République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique

Université de Sfax

École Nationale d'Ingénieurs
de Sfax

Département :
Génie Informatique
et Mathématiques Appliquées

Rapport de stage

présenté à

L'École Nationale d'Ingénieurs de Sfax

filière

Génie Informatique et Mathématiques Appliquées

Réalisé au sein de :

Decade Sfax

par

Najmeddine Maatoug

Projet IA e-commerce : enrichissement et optimisation des données produit

Sous l'encadrement de :

Mme [Nom Encadrante ENIS]	Encadrante académique
M. [Nom Encadrant Decade]	Encadrant industriel

REMERCIEMENTS

Je tiens tout d'abord à exprimer ma profonde gratitude à **Madame [Nom Encadrante ENIS]**, mon encadrante académique à l'École Nationale d'Ingénieurs de Sfax, pour la qualité de son accompagnement scientifique, la pertinence de ses orientations méthodologiques et sa disponibilité constante tout au long de ce travail.

J'adresse mes sincères remerciements à **Monsieur Mohamed SIALA**, Chef de Projet et encadrant industriel chez DECADE Sfax, pour la confiance qu'il m'a accordée dès le premier jour, pour ses conseils techniques avisés, et pour la rigueur bienveillante avec laquelle il a suivi l'avancement du projet.

Je remercie chaleureusement **Monsieur GRAIESS Mahdi**, Gérant de DECADE Tunisie, pour m'avoir offert l'opportunité d'intégrer cette structure, ainsi que **l'ensemble de l'équipe de DECADE Sfax** pour l'accueil professionnel, la bonne humeur quotidienne et les échanges techniques enrichissants qui ont alimenté ma réflexion tout au long du stage.

Mes remerciements s'adressent également au **corps enseignant de la filière Génie Informatique et Mathématiques Appliquées de l'ENIS** pour la solidité de la formation dispensée. Les enseignements reçus en architecture logicielle, en intelligence artificielle et en génie logiciel ont constitué un socle indispensable à la réalisation de ce projet.

Enfin, j'adresse mes plus sincères remerciements à **ma famille et à mes proches** pour leur soutien moral inébranlable, leur patience et leurs encouragements tout au long de ce parcours.

TABLE DES MATIÈRES

INTRODUCTION GÉNÉRALE

La transformation numérique du commerce électronique a profondément reconfiguré la manière dont les entreprises conçoivent, valorisent et diffusent leurs catalogues produits. Dans cet écosystème hautement concurrentiel, la donnée produit n'est plus un simple support descriptif : elle constitue un levier stratégique qui conditionne la qualité de la recherche, la pertinence des recommandations et, au final, la conversion commerciale.

Or, la construction et la maintenance de ces données représentent un défi opérationnel considérable. Les catalogues e-commerce modernes comptent souvent plusieurs dizaines de milliers de références, chacune devant être décrite selon des attributs structurés et cohérents : couleur, matière, catégorie, motif, saisonnalité, description normalisée. Lorsque ces attributs sont incomplets, hétérogènes ou absents, la qualité de la recherche se dégrade, les facettes de navigation deviennent peu fiables, et l'expérience utilisateur s'en trouve directement pénalisée.

Dans le contexte de **DECADE Sfax**, cette problématique prend une dimension opérationnelle forte. Les fiches produit, gérées dans un environnement **SAP Commerce Cloud (Hybris)** et exploitées par le moteur de recherche **Apache Solr**, présentent fréquemment des attributs manquants ou incohérents. L'enrichissement de ces données repose aujourd'hui en grande partie sur des interventions manuelles coûteuses, difficilement scalables et sources d'hétérogénéité.

Le présent projet de fin d'études répond à ce besoin en proposant une solution d'**enrichissement automatique des données produit par intelligence artificielle multimodale**. L'idée directrice consiste à mobiliser un *Large Language Model* capable d'analyser conjointement des images produit et des métadonnées textuelles afin de suggérer des attributs structurés avec un score de confiance associé. Ces suggestions sont ensuite soumises à un pipeline de validation humaine avant d'être injectées dans Hybris et réindexées dans Solr.

Au-delà de l'aspect algorithmique, l'enjeu est d'ordre **industriel** : garantir la fiabilité des données générées, préserver la cohérence avec le modèle de données Hybris existant, maîtriser la traçabilité des décisions de validation, et améliorer l'indexation Solr sans perturber les processus métier en place. Le projet s'inscrit ainsi dans une logique de compromis entre *innovation IA*, *gouvernance de la donnée* et *contraintes de production*.

Ce mémoire est organisé de la manière suivante :

— Le **chapitre ??** présente le cadre général du projet : organisme d'accueil, problématique, objectifs, acteurs impliqués, périmètre et contraintes, ainsi que la démarche méthodologique retenue.

— Le **chapitre ??** formalise les besoins fonctionnels et non fonctionnels, pose les cas d'usage prioritaires, et propose une étude de l'existant : évaluation expérimentale des solutions locales et des APIs LLM candidates, étude comparative des outils PIM concurrents, et état de l'art des technologies mobilisées.

— Le **chapitre 3** détaille la conception de la solution : architecture globale, modèles UML (classes, composants, séquences), choix techniques et stratégie d'intégration dans l'environnement Hybris/Solr.

— Le **chapitre 4** présente la réalisation technique : environnement de développement, implémentation du pipeline d'enrichissement, développement de l'extension Hybris, du dashboard React et de l'API REST.

— Le **chapitre 5** expose les tests et la validation de la solution, au regard des indicateurs de qualité définis en amont.

— Une **conclusion générale** synthétise les apports du projet et trace des perspectives d'évolution.

Ce mémoire poursuit ainsi un fil conducteur clair : *transformer une problématique de qualité de données en une opportunité d'amélioration mesurable de la performance e-commerce, grâce à une IA multimodale intégrée de façon pragmatique et maîtrisée dans l'écosystème Hybris/Solr.*

Chapitre

1 Cadre général du projet

1.1 Introduction

Ce chapitre constitue le socle de référence du mémoire. Il présente l'environnement institutionnel et technique dans lequel s'inscrit le projet, formule la problématique métier, précise les objectifs poursuivis et les acteurs concernés, puis délimite le périmètre de réalisation et la démarche méthodologique retenue.

1.2 Présentation de l'organisme d'accueil

1.2.1 Groupe DECADE

DECADE est une société de services spécialisée dans le développement de sites e-commerce. Fondée et implantée à Paris, elle accompagne des marques et retailers dans la conception, l'intégration et l'optimisation de leurs plateformes de vente en ligne, avec une expertise centrale sur **SAP Commerce Cloud (Hybris)**, l'un des CMS e-commerce les plus déployés dans les environnements retail de grande envergure.

1.2.2 DECADE Tunisie — filiale de Sfax

DECADE Tunisie est la filiale offshore du groupe. Son siège social est situé à Monastir, et ses activités de développement sont principalement portées par l'entité de **Sfax**, localisée dans l'Immeuble Focus sur la Route Gremda Ceinture Bourguiba. DECADE Tunisie opère en tant que centre de compétences techniques dédié aux projets e-commerce du groupe, avec un positionnement fort sur la qualité logicielle, la performance des solutions Hybris et l'optimisation des parcours utilisateurs.

L'équipe accompagne au quotidien des projets à forte complexité technique, couvrant notamment :

— l'intégration et la personnalisation d'extensions SAP Hybris (Java/Spring ;

— la gestion des catalogues produit et des flux de données ;

— l'optimisation de la recherche via **Apache Solr** ;

1.2.3 Cadre académique du stage

Ce travail s'inscrit dans le cadre du Projet de Fin d'Études de la filière **Génie Informatique et Mathématiques Appliquées** de l'École Nationale d'Ingénieurs de Sfax (ENIS), sur la période du 02 février au 31 juillet 2026. L'encadrement industriel est assuré par **Monsieur Mohamed SIALA**, Chef de Projet chez DECADE Sfax, et l'encadrement académique par **Madame [Nom Encadrante ENIS]** de l'ENIS.

1.3 Problématique

1.3.1 Contexte opérationnel

Dans un catalogue e-commerce volumineux, chaque fiche produit doit exposer un ensemble d'attributs structurés et cohérents afin de garantir la qualité de la navigation, des filtres et du moteur de recherche. Or, dans l'environnement actuel de DECADE, l'enrichissement de ces attributs repose largement sur des interventions humaines manuelles, avec plusieurs conséquences mesurables :

— **Charge opérationnelle élevée** : les équipes métier consacrent un temps considérable à la saisie et à la correction des données produit ;

— **Hétérogénéité de saisie** : valeurs redondantes, variantes lexicales non normalisées et omissions fréquentes dégradent la cohérence du catalogue ;

— **Retard d'indexation** : des attributs utiles pour Solr (couleur, matière, saisonnalité) sont absents ou incorrects, appauvrissant les facettes et les filtres dynamiques ;

— **Impact sur la conversion** : une recherche imprécise réduit la visibilité des produits pertinents et détériore l'expérience utilisateur finale.

1.3.2 Formulation de la problématique

La problématique centrale du projet peut être formulée de la façon suivante :

Comment automatiser l'enrichissement des attributs produit à partir de données multimodales (images, texte, métadonnées), tout en assurant la qualité, la traçabilité et l'intégration native dans l'écosystème SAP Hybris / Apache Solr ?

1.4 Objectifs du projet

1.4.1 Objectif principal

Concevoir et développer une solution intelligente d'enrichissement automatique des attributs produit par IA multimodale, intégrée nativement à **SAP Commerce Cloud (Hybris)** et exploitable dans **Apache Solr** pour améliorer la qualité de la recherche e-commerce.

1.4.2 Objectifs spécifiques

— Extraire automatiquement des attributs structurés : couleur dominante et secondaire, matière/texture, type de produit, motif, saisonnalité et description normalisée ;

— Affecter un score de confiance par attribut pour guider les décisions de validation ;

— Mettre en place un espace de pilotage humain (validation, correction, rejet) pour maîtriser la qualité avant injection ;

— Injecter les attributs validés dans Hybris avec une logique de mapping, de gestion de statuts et de mécanismes de reprise en cas d'erreur ;

— Alimenter Apache Solr pour améliorer facettes, filtres dynamiques et pertinence des résultats de recherche ;

— Exposer les données enrichies via une API REST consommable par d'autres composants du système.

1.4.3 Indicateurs cibles

Afin d'évaluer objectivement l'apport de la solution, les indicateurs suivants sont retenus :

— **Taux de complétion** : proportion d'attributs renseignés avant et après enrichissement ;

— **Taux de validation manuelle** : part des suggestions acceptées par les gestionnaires produit ;

— **Score de confiance moyen** : niveau moyen de fiabilité par attribut généré par le LLM ;

— **Impact sur la recherche** : amélioration des facettes disponibles et de la précision perçue dans Solr.

1.5 Acteurs et interactions principales

Le système met en jeu des acteurs humains et techniques complémentaires dont les rôles sont définis comme suit :

— **Gestionnaire produit** : consulte les suggestions d'enrichissement via le dashboard, valide, corrige ou rejette les attributs proposés ;

— **Administrateur Hybris** : configure l'extension, supervise le mapping des attributs vers le modèle de données Hybris et gère les versions d'enrichissement ;

— **Service IA multimodal (LLM)** : analyse les images et les métadonnées produit, génère les attributs enrichis avec score de confiance ;

— **Apache Solr** : consomme les attributs validés et les indexe pour améliorer la recherche facettée ;

— **Client final** : bénéficie indirectement d'une navigation plus précise et de résultats de recherche plus pertinents.

1.6 Périmètre et contraintes

1.6.1 Périmètre fonctionnel

Le périmètre du PFE couvre les briques suivantes :

— enrichissement automatique des attributs prioritaires (couleur, matière, catégorie, motif, saisonnalité, description) ;

— validation humaine via un dashboard React dédié ;

— intégration des attributs validés dans l'extension SAP Hybris ;

— réindexation des nouveaux champs dans Apache Solr ;

— exposition des données enrichies via API REST.

1.6.2 Hors périmètre

Les éléments suivants sont explicitement exclus du périmètre de ce PFE :

— intégration avec des systèmes PIM externes (Akeneo, Salsify, etc.) ;

— déploiement en environnement de production (livraison en environnement staging) ;

— génération de contenus visuels ou vidéo produit ;

— développement d'une fonctionnalité de recommandation produit.

1.6.3 Contraintes de réalisation

Les contraintes majeures identifiées sont :

— **Compatibilité Hybris** : respect du modèle de données existant (`items.xml`, ServiceLayer, FlexibleSearch) ;

— **Maîtrise des coûts LLM** : optimisation du traitement batch pour contenir la consommation de tokens ;

— **Robustesse des échanges API** : gestion des erreurs, mécanismes de retry et traçabilité complète ;

— **Qualité éditoriale** : cohérence et normalisation des attributs générés, notamment dans une perspective multilingue (FR, EN, DE).

1.7 Démarche méthodologique et planification

1.7.1 Méthodologie adoptée : Scrum adapté

Le projet s'appuie sur une démarche **Agile Scrum adaptée** au contexte d'un stage PFE. Le périmètre et les priorités fonctionnelles sont fixés en début de projet avec l'encadrant industriel, puis l'implémentation est organisée en **sprints de deux semaines** permettant des livraisons intermédiaires régulières, des revues et des ajustements.

Cette approche offre plusieurs avantages dans notre contexte :

— capacité d'adaptation continue aux retours des parties prenantes ;

— détection précoce des risques techniques (latence LLM, compatibilité Hybris) ;

— livraisons progressives qui facilitent la validation métier à chaque étape.

Les réunions de suivi hebdomadaires avec l'encadrant industriel jouent le rôle de *sprint reviews* informelles, et le tableau de bord de planification assure la visibilité sur l'avancement.

1.7.2 Planification générale

Le Tableau **??** présente la répartition chronologique des phases du projet sur la durée du stage.

TABLE 1.1 – Planification générale du projet (février – juillet 2026)

#	Phase	Semaines	Livrables principaux
1	Compréhension & Cadrage	S1 – S2	Rapport de cadrage, étude de l'existant, besoins formalisés
2	Spécifications & Conception	S3 – S6	Diagrammes UML, architecture cible, maquettes Dashboard
3	Développement – Sprint 1	S7 – S8	POC LLM, pipeline d'extraction d'attributs
4	Développement – Sprint 2	S9 – S10	Extension Hybris, mapping, gestion des statuts
5	Développement – Sprint 3	S11 – S12	Dashboard React (validation, comparaison, KPIs)
6	Développement – Sprint 4	S13 – S14	Intégration Solr, API REST, gestion multilingue
7	Tests & Validation	S15 – S16	Plan de tests, mesure des indicateurs, corrections
8	Documentation & Rédaction	S17 – S22	Mémoire final, document d'exploitation, démonstration

1.8 Conclusion

Ce chapitre a posé les fondations du projet : contexte entreprise, problématique, objectifs, acteurs, périmètre, contraintes et démarche méthodologique. Il constitue le référentiel de cadrage sur lequel s'appuient l'ensemble des choix d'analyse et de conception présentés dans la suite du rapport. Le chapitre suivant approfondit les besoins, étudie les solutions existantes et pose l'état de l'art des technologies retenues.

Chapitre

2

Étude de l’existant

2.1 Introduction

Ce chapitre présente l'existant sur lequel s'appuie et s'intègre le projet. Il se structure en deux volets complémentaires. Le premier analyse Apache Solr, déjà opérationnel dans l'environnement SAP Commerce de DECADE, en identifiant ses limites directement liées à la qualité des données produit et en montrant comment l'enrichissement LLM les adresse. Le second évalue expérimentalement les solutions LLM disponibles (locales, gratuites, GPU cloud, payantes) afin de retenir le modèle le mieux adapté au pipeline d'enrichissement des fiches produit.

2.2 Volet 1 — Apache Solr dans SAP Commerce (Existant interne)

2.2.1 Rôle et intégration

Apache Solr est le moteur de recherche et d'indexation déjà intégré nativement dans l'environnement SAP Commerce Cloud (Hybris) de DECADE. Basé sur Apache Lucene, il assure trois fonctions critiques pour le catalogue e-commerce : la recherche produit full-text, les filtres de navigation par facettes, et l'auto-complétion.

Son rôle dans l'architecture est complémentaire à la base de données Hybris : là où Hybris stocke la source de vérité (les données produit), Solr maintient un index dédié, optimisé pour répondre aux requêtes utilisateurs en quelques millisecondes.

2.2.2 Flux d'indexation

Le processus d'indexation se déroule en cinq étapes séquentielles :

1. Les données produits sont stockées dans la base SAP Commerce ;
2. Un processus d'indexation les extrait et les transforme en documents Solr ;
3. Ces documents sont persistés dans l'index Solr ;
4. Lorsqu'un utilisateur effectue une recherche, la requête est transmise à Solr ;

5. Solr retourne les résultats avec les facettes associées.

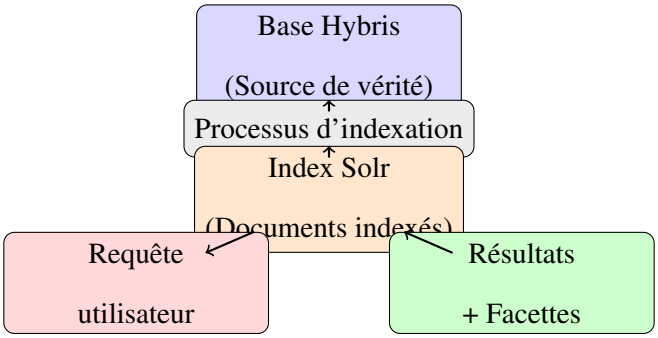

FIGURE 2.1 – Flux d'indexation et de recherche Solr dans SAP Commerce

2.2.3 Limitations structurelles de Solr

Bien qu'opérationnel, Solr présente six limitations qui dégradent directement la qualité de recherche et motivent le présent projet :

2.2.3.1 Limitation 1 — Modèle lexical et dépendance aux mots-clés

Solr analyse les tokens (mots) présents dans la requête et les rapproche des mots présents dans l'index. Il ne comprend pas la signification ou l'intention derrière la requête.

> *Exemple : Requête « best vacuum for pet hair » -> Solr analyse [best][vacuum][pet][hair] -> ne comprend pas « aspirateur adapté aux poils d'animaux ».*

2.2.3.2 Limitation 2 — Absence de compréhension sémantique

Solr ne possède pas de mécanisme natif de recherche sémantique. Il ne peut ni décoder l'intention utilisateur, ni gérer les synonymes implicites, ni classer les résultats par pertinence de sens.

2.2.3.3 Limitation 3 — Personnalisation du ranking limitée

En configuration standard, Solr retourne les mêmes résultats pour tous les utilisateurs effectuant la même requête. Les moteurs modernes s'appuient sur le machine learning pour personnaliser le ranking (historique de navigation, préférences). Solr exige des règles manuelles de boosting.

2.2.3.4 Limitation 4 — Complexité de configuration et maintenance

Améliorer la pertinence nécessite des interventions techniques importantes : règles de boosting, dictionnaires de synonymes, analyseurs linguistiques. Cette complexité accroît les coûts de maintenance et exige une expertise pointue.

2.2.3.5 Limitation 5 — Dépendance à l'indexation complète

Lors d'un Full Index, l'index existant est remplacé, ce qui peut créer une indisponibilité temporaire de la recherche. Sur un catalogue de grande taille, l'opération est longue et coûteuse en ressources.

2.2.3.6 Limitation 6 — Qualité des résultats tributaire de la qualité des données

C'est la limitation la plus directement liée au projet : si les attributs produit sont incomplets ou hétérogènes dans Hybris, l'index Solr reproduit ces lacunes. Les facettes (couleur, matière, saisonnalité) ne peuvent exister que si les attributs correspondants sont renseignés dans la source.

2.2.4 Illustration concrète — Cas Jacadi

Un exemple réel illustre ces limitations : dans le catalogue Jacadi (vêtements enfants), la recherche du terme « table » retourne des produits vestimentaires. Plusieurs mécanismes combinés l'expliquent :

— **Free Text Search** : Solr indexe la description complète des produits. Une description comme « idéal pour dîner à table en famille » contient le token « table » ;

— **Fuzzy Search** : la recherche approximative peut matcher « table » sur « tableau » ou « tablette » ;

— **Synonymes configurés** : si le backoffice définit « table ↔ furniture », la requête s'étend à « table OR furniture » ;

— **Tokenisation** : un analyseur mal configuré peut extraire « table » depuis « comfortable ».

2.2.5 En quoi l'enrichissement LLM améliore Solr

L'enrichissement LLM ne remplace pas Solr, il le complète en alimentant l'index avec des attributs structurés, normalisés et fiables.

2.2.5.1 Amélioration des facettes

En générant des attributs enrichis (couleur, matière, occasion, style, saisonnalité), le LLM crée les champs manquants qui deviendront des facettes exploitables dans Solr.

TABLE 2.1 – Comparaison avant/après enrichissement LLM dans Solr

	Avant enrichissement	**Après enrichissement LLM**
Attributs indexés	name, description (texte libre)	couleur, matière, catégorie, motif, saison, occasion
Facettes disponibles	Aucune ou très limitées	Navigation par couleur, matière, saison, occasion
Pertinence Solr	Dépend du texte libre uniquement	Filtrée sur attributs précis et normalisés

2.2.5.2 Flux d'intégration : de l'enrichissement LLM à Solr

Le projet s'inscrit dans un flux linéaire : le LLM génère les attributs manquants, ces attributs sont validés par un gestionnaire produit, injectés dans Hybris, puis réindexés dans Solr. La Figure **??** illustre ce flux.

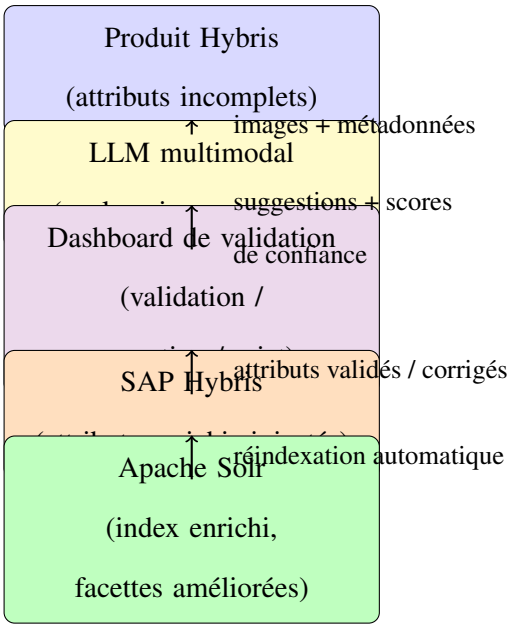

FIGURE 2.2 – Flux d'enrichissement des attributs par LLM multimodal et injection dans Solr

2.3 Volet 2 — Évaluation des solutions LLM disponibles

2.3.1 Objectif et protocole

Avant de retenir un modèle pour la production, une phase d'exploration comparative a été menée. L'objectif est de qualifier chaque approche sur trois critères : capacité à traiter des scénarios réalistes, stabilité, et viabilité pour un usage productif continu.

Les tests ont été conduits sur quatre niveaux de complexité croissante :

1. **Texte seul** : description produit sans image ;
2. **Image unique** : une seule image produit encodée en base64 ;
3. **Multi-images** : 3 à 4 vues du même produit ;
4. **Images + métadonnées** : multi-images + attributs existants + description textuelle.

Ce dernier scénario représente le cas d'usage réel du projet et génère un contexte volumétrique élevé.

2.3.2 Axe 1 — Exécution locale via Ollama

2.3.2.1 Présentation

Ollama est un outil open source permettant d'exécuter des LLMs directement en local, sans API externe. Ses avantages théoriques sont la confidentialité des données et l'absence de quota. Trois modèles multimodaux ont été testés : Gemma 4 (Google DeepMind), Qwen-VL (Alibaba) et Moondream (open source).

2.3.2.2 Résultats

L'exécution locale révèle une limite structurelle matérielle : sans GPU dédié, le traitement repose entièrement sur le CPU, causant une latence prohibitive, une saturation de la RAM par les images encodées en base64, et des instabilités répétées sur les cas multi-images.

Nom	Statut	100% Processeur	76% Mémoire	11% Disque	0% Réseau	Proces
Interpréteur de commandes Wi...		96,5%	10 364,1 ...	0 Mo/s	0 Mbits/s	

FIGURE 2.3 – Saturation CPU à 96,5% lors d'inférence Ollama locale

```
time=2026-04-15T12:27:41.402+02:00 level=WARN source=runner.go:187 msg="truncating input prompt"
 keep=4 new=4096
[GIN] 2026/04/15 - 12:32:54 | 200 |          5m41s |       127.0.0.1 | POST      "/api/chat"
```

FIGURE 2.4 – Erreur d'inférence Ollama — modèle trop lourd pour la session GPU actuelle

2.3.2.3 Verdict

Non viable pour la production. Exploitable uniquement pour exploration et validation de POC en dehors de toute contrainte de temps de réponse.

2.3.3 Axe 2 — APIs gratuites (OpenRouter, Groq)

2.3.3.1 OpenRouter (Free Tier)

OpenRouter est une plateforme d'agrégation d'APIs LLM. Le tier gratuit impose une fenêtre de contexte réduite.

— **Gemma 4 (Google)** : réussit sur texte et image unique ; context overflow sur multi-images et images + métadonnées ;

— **Nemotron Nano 12B 2 VL (NVIDIA)** : réussit jusqu'au multi-images, mais context overflow sur le scénario complet ;

— **Qwen 3.6 Plus (Alibaba)** : crash dès la première image ; erreurs HTTP 429 dès le premier appel.


```
You, last week | 1 author (You)
[
  {
    "productId": "2045144_895",
    "productVersion": 95,
    "imagesUsed": [
      "C:\\Users\\NejmeddinBENMAATOUG\\Desktop\\PFE\\working on models to test\\resources\\images\\2045144_895\\1.jpg",
      "C:\\Users\\NejmeddinBENMAATOUG\\Desktop\\PFE\\working on models to test\\resources\\images\\2045144_895\\2.jpg",
      "C:\\Users\\NejmeddinBENMAATOUG\\Desktop\\PFE\\working on models to test\\resources\\images\\2045144_895\\3.jpg",
      "C:\\Users\\NejmeddinBENMAATOUG\\Desktop\\PFE\\working on models to test\\resources\\images\\2045144_895\\4.jpg"
    ],
    "error": "429 Client Error: Too Many Requests for url: https://openrouter.ai/api/v1/chat/completions"
  }
]      You, last week • openRouter free multimodal models : I test thei…
```


FIGURE 2.5 – Erreur HTTP 429 sur OpenRouter — quota dépassé sur multi-images

2.3.3.2 Groq (Free Tier)

Groq propose une inférence ultra-rapide via des puces LPU propriétaires. Le tier gratuit est limité à **12 000 tokens par minute (TPM)**.

Pour un seul produit avec 4 images et ses métadonnées, le test a consommé **46 214 tokens**, dépassant le quota d'un facteur 3,8×. Le serveur retourne systématiquement :

```
Error 413: Request Too Large for 'llama-3.3-70b-versatile'
Limit 12,000 TPM | Requested 46,471 tokens
-> rate_limit_exceeded
```

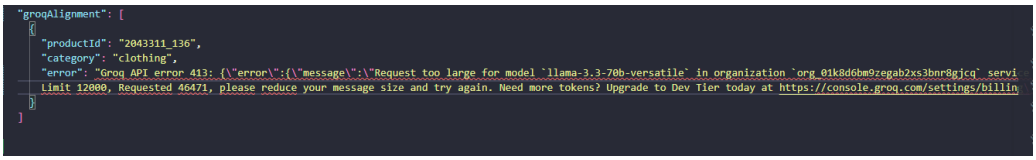

FIGURE 2.6 – Dépassement du quota TPM Groq (×3,8 sur un seul produit avec métadonnées)

2.3.3.3 Verdict

Utile pour le prototypage rapide uniquement. La contrainte de quotas rend ces plateformes incompatibles avec un flux d'enrichissement e-commerce soutenu.

2.3.4 Axe 3 — GPU cloud (Google Colab)

Pour contourner les limites matérielles locales, des tests ont été conduits sur Google Colab avec GPU gratuit. Deux modèles lourds ont été évalués :

2.3.4.1 Qwen 3.5 27B

Latence : **12 minutes** pour une phrase simple. Consommation GPU : 13,3 Go sur 15,0 Go disponibles. Marge de sécurité insuffisante pour un traitement en lot.

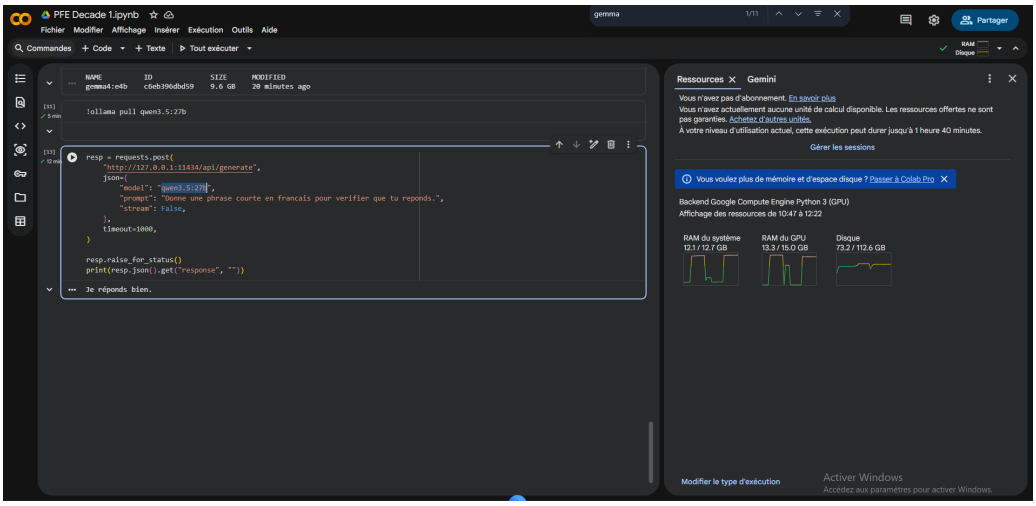

FIGURE 2.7 – Qwen 3.5 27B sur Colab — latence de 12 min, RAM GPU à 13,3/15 Go

2.3.4.2 Gemma 31B

Latence : **5 minutes**. Consommation GPU : 14,0 Go sur 15,0 Go, frôlant l'erreur Out Of Memory. RAM système à 11,5 Go.

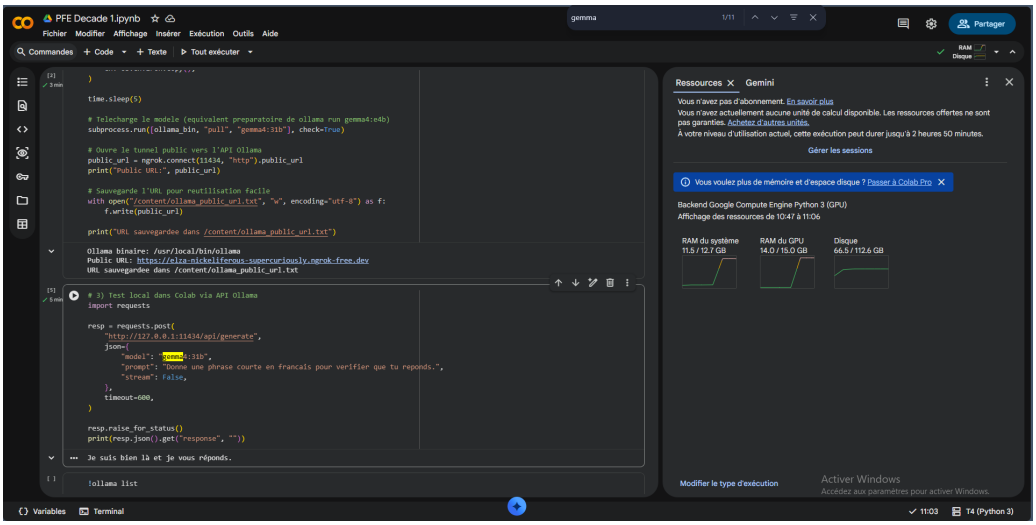

FIGURE 2.8 – Gemma 31B sur Colab — saturation GPU critique à 14,0/15 Go

2.3.4.3 Verdict

Non viable pour la production. La latence prohibitive et les marges d'erreur minimales rendent ces configurations inadaptées à un pipeline batch.

2.3.5 Axe 4 — APIs payantes (candidats production)

Les résultats des trois axes précédents orientent naturellement vers les APIs payantes. Une comparaison qualitative des candidats les plus crédibles est présentée dans le Tableau **??**.

TABLE 2.2 – Comparaison des APIs payantes candidates pour la production

Solution	Forces	Latence	Coût	Risque principal
GPT-4o (OpenAI)	Qualité multimodale élevée, JSON mode natif, robustesse prouvée	Faible–moyenne	Élevé	Coût à l'échelle, politique de données
Gemini 2.5 Pro (Google)	Contexte très large, bonne analyse visuelle	Moyenne	Moyen–élevé	Quotas selon plan, RGPD
Claude Sonnet (Anthropic)	Rigueur sémantique, sortie JSON très fiable	Faible–moyenne	Élevé	Coût, intégration selon stack

Ces solutions répondent aux exigences critiques du projet : robustesse, continuité de service, support des scénarios multi-images, et gestion contrôlée des quotas. Le choix final sera confirmé en accord avec la politique de gouvernance des données de DECADE.

2.3.6 Tableau comparatif global

Le Tableau **??** résume l'ensemble des tests conduits selon les quatre scénarios de complexité.

TABLE 2.3 – Tableau comparatif global des solutions LLM testées

Plateforme	Modèle	Texte	1 image	Multi-img	Img+Méta	Verdict
Ollama (local)	Gemma 4	**Lent** 2 min	**Lent** 6 min	**KO**	**KO**	Non viable
	Qwen-VL	**Lent** 2 min	**Lent** 6 min	**KO**	**KO**	Non viable
	Moondream	**Lent** 3 min	**Lent** 5 min	**KO**	**KO**	Non viable
OpenRouter Free	Gemma 4	**OK**	**OK**	**KO** overflow	**KO** overflow	Proto only
	Nemotron 12B	**OK**	**OK**	**OK**	**KO** overflow	Proto only
	Qwen 3.6+	**OK**	**KO** crash	**KO**	**KO**	Non viable
Groq Free	LLaMA 3.3 70B	**OK**	**OK**	**OK**	**KO** ($\times$3,8)	Non viable
	Groq Compound	**OK**	**OK**	**OK**	**KO** ($\times$3,8)	Non viable
Google Colab GPU	Qwen 3.5 27B	**OK**	**OK** 12 min	**Lent**	**Lent**	Exploration
	Gemma 31B	**OK**	**OK** 5 min	**Lent**	**Lent**	Exploration
APIs Payantes	GPT-4o	**OK**	**OK**	**OK**	**OK**	Production
	Gemini 2.5 Pro	**OK**	**OK**	**OK**	**OK**	Production
	Claude Sonnet	**OK**	**OK**	**OK**	**OK**	Production

2.3.7 Synthèse de décision

L'étude expérimentale établit une trajectoire claire :

— **Local (Ollama)** : exploration et POC, latence prohibitive sans GPU dédié ;

— **APIs gratuites (OpenRouter, Groq)** : prototypage rapide, quotas incompatibles avec un flux continu ;

— **GPU cloud gratuit (Colab)** : investigations techniques, latence et marges inacceptables ;

— **APIs payantes** : seule option réaliste pour la robustesse et la continuité de service requises.

La stratégie retenue est **hybride** : utiliser les offres gratuites pour le prototypage initial et l'ajustement des prompts, puis basculer vers une API payante pour la validation industrielle et le déploiement progressif.

2.4 Conclusion

Ce chapitre a établi l'état de l'existant sur deux axes. D'une part, Apache Solr — déjà opérationnel dans le projet — présente six limitations structurelles, dont la principale est la dépendance directe à la complétude des attributs produit dans Hybris : des attributs manquants ou incohérents produisent un index Solr appauvri, des facettes peu fiables et une pertinence de recherche dégradée. L'enrichissement LLM adresse ce verrou en alimentant Hybris — et donc Solr — avec des attributs structurés, normalisés et validés. D'autre part, l'évaluation comparative des solutions LLM démontre que seules les APIs payantes offrent le niveau de robustesse requis pour industrialiser ce pipeline. Ces deux conclusions guident directement les choix de conception présentés dans le chapitre suivant.